Temco Platform

Temco Platform — architecture · built Nov 12, 2026

Temco Platform — architecture

Temco Platform is the **authoring** application: a desktop app built on a VS Code fork where documentation is written, organised into projects, built into a website — plus a whole-guide PDF and a Windows help file — and published to Temco Learn, the site readers open.

Everything Temco builds is either **the shell** (application behaviour) or **a module** (a feature).

```text
write → build → add → make the downloads → publish → GitHub Pages
```

Act	Where it happens	What it produces
write	the project's `docs/` folder, in the editor	Markdown pages and a registry
build	the project folder, by its own scripts or the built-in generator	`dist/site`
add	the site clone, by **Add This Build to the Site**	`public/imported/<id>/<version>/`
make the downloads	the site clone, by **Make the Downloads**	`public/pdf/`, `public/chm/`, the download manifest
publish	the site clone, by **Publish**	one commit on `main`; CI builds and deploys it

Start here	For
introduction.md	**using** the platform: the flow above, step by step, with a file to look at after each step
this file	**working on** the platform: the shell, the modules, the rules that keep them apart

Repositories

Repository	Is
`temco-platform`	the application: the shell and every module
`temco-learn`	the published documentation site (Temco Learn)

The application and all its modules live in one repository. `temco-learn` is separate because it is a published artifact, not a module.

Layout

TEXT

```
temco-platform/
├── product.json                    product name, icons, default settings, policy
├── resources/                      logos, installer art, application icons
├── src/
│   ├── vs/                         VS Code — never edited
│   └── temco/                      the shell
├── extensions/
│   ├── temco-docs/                 modules ([../modules/README.md](./modules/README.md
│   ├── temco-ai/
│   └── temco-*/                    one folder per module
├── docs/                           this documentation
└── build/  cli/  scripts/  test/  remote/     VS Code — never edited
```

Where code goes

Path	Contents	Edited by
`src/vs/**`, `build/**`, `cli/**`, `scripts/**`, `test/**`, `gulpfile*`	VS Code	nobody
`product.json`, `resources/**`, `package.json`	product identity and policy	us
`src/temco/**`	the shell	us
`extensions/temco-*/**`	the modules	us
`docs/**`	documentation	us

The rule: every change Temco makes is a new file in a new folder. A change to VS Code's own files is a decision, not a habit.

The shell

```text
src/temco/
├── temco.contribution.ts   the single entry point the workbench loads
├── menus/                  top-level menus and the items in them
├── branding/               About, splash, identity behaviour
├── startup/                welcome, first run, default layout
├── services/               services the modules consume
└── platform/               plumbing shared by the folders above
```

The shell owns the application's identity, its startup, its top-level menus, its default layout and the services modules share. It owns no feature. A feature that only works inside the shell belongs to a module, not to the shell.

Entry points

Two additive lines in VS Code's files: entry-points.md.

File	Addition
`src/tsconfig.json`	`src/temco/**` added to `include`
`src/vs/workbench/workbench.desktop.main.ts`	one import of `src/temco/temco.contribution.ts`

Modules

A module is a real extension. Its shape, its contract and its register: ../modules/README.md.

```text
extensions/temco-<module>/
├── README.md               purpose, identifiers, what it owns
├── package.json            id, engine range, commands, views, settings, activation
├── tsconfig.json           extends ../tsconfig.base.json
├── src/
│   └── extension.ts        the only activation entry: register, wire, dispose
├── media/                  icons, webview css and js
├── prompts/                model instructions, where a module needs them
└── scripts/                module-local gates
```

A module's documentation lives in `docs/temco-platform/modules/temco-<module>/` (../modules/README.md), never in a `docs` folder inside the module.

State	Contents	Build effect
reserved	`README.md` only	nothing is compiled and nothing ships
active	manifest, tsconfig, `src/`	compiled and shipped as a built-in extension

Only `temco-docs` is active today.

Nothing needs registering for a module to build or ship: the build picks up every `extensions/*/package.json`, the root install brings in each folder's own dependencies, and web builds skip anything that is not a web extension.

Publishing your documentation

New to the platform? Start here: from an empty folder to a page on Temco Learn, in nine steps, each with a file to look at before you go on — introduction.md. The same document carries the rest of what a project can do — sources, the editor, validate, release, export, translate — in *Beyond the flow*.

Naming

All identifiers, and the rename map applied at the move: naming.md.

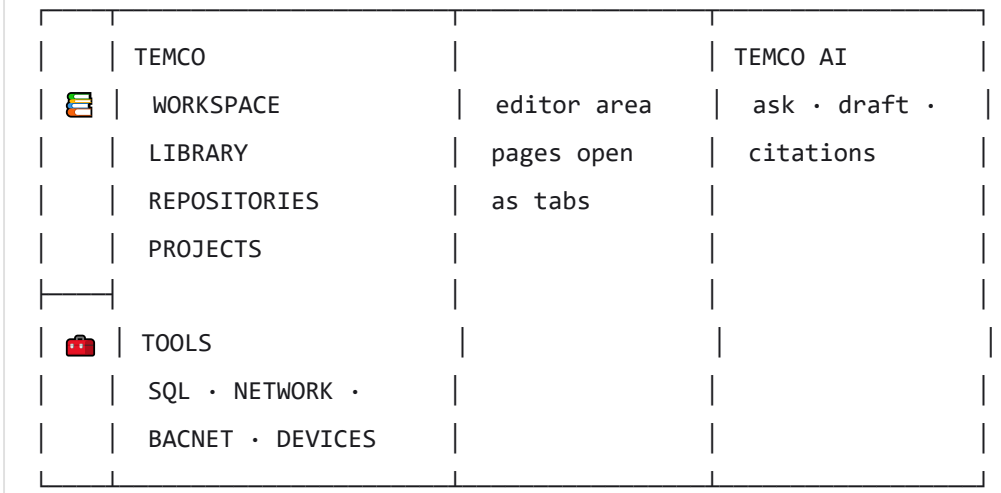
The catalog

The platform's memory — the tables `temco-store` keeps, what each one is for, and every setting a module declares: catalog.md.

UI layout

Two activity-bar icons, the editor in the middle, the AI on the right.

TEXT



Container id	Title	Declared by	Filled by
`temco`	Temco	the docs module now, the shell later	docs, repositories, projects
`temcoTools`	Tools	the first tools module	SQL, network, BACnet, devices
`temcoAi`	Temco AI	the AI module	the AI panel, secondary side bar

A container with no views is not shown, so a container may be declared before its views exist.

Surface	Contributed by	Appears as
view container	module or shell	an icon in the activity bar
view	module	a section in a sidebar
secondary-side-bar view	module	a panel on the right
status bar item	module	an item in the bottom bar
diagnostics	module	rows in Problems
webview editor	module	a tab in the editor area

Rules

1. VS Code's own source is never edited; the two entry points are the complete list.
2. Every change is a new file in a new folder.
3. A module must also work in an ordinary VS Code window — it uses the published `vscode` API only, nothing shell-specific.

4. Modules talk through commands, settings, files and events; never by importing one another's source.
5. `temco-docs` owns the catalog and every page write.
6. Read-only by default. The application writes only in the five documented cases, each behind its own command and a confirmation naming the path or the value.
7. Nothing is ever written to a source repository.
8. The only thing the application pushes is the site: a publish commits `temco-learn` and pushes `main`, and its CI deploys to Pages (introduction.md). A project's own build is never deployed from here.
9. A build engine sits behind the content-package contract (content-package.md); nothing outside the engine's own folder names it.
10. Runtimes and models are downloaded on request into the user's own folder; the engine runs on loopback only.
11. A guide is a promise, so the docs are checked against the manifests: every `temco*.*` id in backticks must be a declared command, a registered command or a setting, every relative link and `#anchor` must land, `json` examples must parse, screenshot slots must be numbered `00, 01, 02 …`, and `## Step N` must not skip. Rename a command and the guide fails (introduction.md is the guide; `npm run check-temco-docs` is the check).

Checks

Four commands. `check-temco` runs as part of `npm run compile`, so a stale guide or a broken button stops the build; the others are run when the thing they cover changes.

command	what it holds
`npm run check-temco`	the module boundaries, every button's command id against the manifests, the shared components, and `check-temco-docs`
`npm run check-temco-docs`	what the pages *say*: ids, links, anchors, `json`, screenshot slots, step numbers (docs.mts)
`npm run test-temco`	every suite in test/temco: the pages, the store on a real database, where a project's folder goes, the scripts a build runs, the site publish order, the publish guard, the settings rows, starters and imports, and the environment the tool server is given
`npm run test-temco-e2e`	the same application, running: every module activates, every declared command is registered, the catalog opens in the window's own profile, and a Settings-page setting round-trips through the editor's own configuration service — then **a first run, in order**: nothing configured, a folder chosen, a project made from the starter, that project's own scripts building it with a real `node`, the platform finding that build by itself, the build going onto the site, and the project still there afterwards

`test-temco` compiles the six extensions it loads, because the code it tests imports `vscode` and has to be stubbed the way the running editor would call it — one process per suite, since two suites cannot both be the stub that answers `require('vscode')`. The publish cases need a site clone; with none on the machine they skip and say so, and `TEMCO_SITE` points them at one elsewhere.

`test-temco-e2e` needs a build (`npm run compile`) and opens a real window in a throwaway profile. It is the only check that can catch what the stubs cannot: the breadcrumbs row wrote `workbench.breadcrumbs.enabled`, which this editor does not register, so the checkbox threw in a real window while every stubbed suite stayed green. Its `firstrun.case.js` is the one place the modules meet: it drives the real compiled modules in the order the guide describes, so a change in one module that breaks the next one's assumption fails there and nowhere else. `TEMCO_E2E_KEEP=1` keeps the profile for its logs, and `TEMCO_E2E_TIMEOUT_MS` bounds the run.

A suite is one file in test/temco that ends by handing its cases to `standalone()`; the runner finds the rest. A case is one `test('what must be true', () => { … })` line, and the failure it reports is the assertion's own message, so the case and the reason arrive together.

Branches

This repository is a fork of `microsoft/vscode`, so the two jobs are kept on two branches.

Branch	Is	Receives
`main`	a **mirror** of `upstream/main` — no Temco file, no Temco commit	upstream only, fast-forwarded
`t3000-platform`	**the product**: the shell, every module, the product identity	every Temco change, and each upstream sync

`main` is never edited and never carries a release: a tag on it would build an app with none of this product in it. Everything Temco is committed on `t3000-platform`, with `main` as its merge base.

Set `t3000-platform` as the repository's default branch (*Settings → General → Default branch*) — it is `main` today. GitHub only reads `workflow_dispatch` from the default branch, so while `main` is default the release workflow has no *Run workflow* button at all, and a clone lands on upstream's source rather than on the product. A ruleset blocking direct pushes to `main` is worth adding at the same time: the release guards stop a *release* from `main`, not a *commit* to it.

Syncing upstream into `main`:

```shell
git fetch origin --prune && git fetch upstream --prune
git switch main
git reset --hard upstream/main
git push --force-with-lease origin main
```

Bringing that merge into the product — a **merge**, never a rebase: the shared commits rewritten on every upstream release would make `t3000-platform` unusable for anyone who has cloned it.

```shell
git switch t3000-platform
git merge main            # conflicts land on package.json (version), product.json, resc
```

Releasing

A release is a **tag on `t3000-platform`**. GitHub builds the installers from it and attaches them to a draft Release named after the tag. The workflow is `.github/workflows/release.yml`.

The first release, in order

1. **Commit what is pending on `t3000-platform`** — the workflow, the root `README.md` and this page — and push it.

2. **Make `t3000-platform` the default branch** — *Settings → General → Default branch*. Until this is done the workflow has no *Run workflow* button. Add a ruleset blocking direct pushes to `main` while you are there.
3. **Leave `package.json`'s `version` alone.** It carries the VS Code base (`1.144.0` today) and is what the application reports to extensions that are not built in, where `engines.vscode` is enforced: a lower number, or a prerelease suffix, is refused. A Temco build number is the patch digit (`1.144.1`). What an upstream sync does to the number is in the root README.
4. **Tag that commit and push the tag.**

   SHELL

   ```
   git tag -a v<version> -m "Temco Platform <version>"
   git push origin v<version>
   ```

   Actions → Release then builds on three runners and leaves a draft Release. Check the assets, publish it.

Cutting a release

SHELL

```
# 1. the version the installer will report, and the tag, are the same number
#    edit "version" in package.json
git commit -am "Temco Platform <version>"
git push origin t3000-platform

# 2. the tag is the release
git tag -a v<version> -m "Temco Platform <version>"
git push origin v<version>
```

Then open *Actions → Release*. Three runners build in parallel — Windows x64, Linux x64, macOS arm64 — and a **draft** Release named `Temco Platform <version>` is created with:

Asset	Is
`TemcoPlatform-win32-x64-<version>.zip`	the unpacked app: run `Temco Platform.exe`
`*Setup*.exe`	the per-user Windows installer, no admin rights needed
`temco-platform_<version>_amd64.deb`	the Debian/Ubuntu package
`temco-platform-linux-x64-<version>.tar.gz`	the unpacked Linux app
`TemcoPlatform-darwin-arm64-<version>.zip`	Apple silicon only

Check the assets, then publish the draft. Nothing is released while it is a draft.

Two guards, and why they exist

Both fail in seconds, before the 40-minute build:

1. **The tag must equal `v<version>`** in `package.json`. The installer takes its version from `package.json`, not from the tag, so a release called `v1.2.3` that installs itself as `1.144.0` is one nobody can support.
2. **The tag must be on `t3000-platform`.** `main` mirrors upstream, so a tag there would build an application with none of this product in it.

Rebuilding a tag

Run the workflow by hand (*Actions → Release → Run workflow*) with the tag's name. Nothing is re-tagged and the draft of a failed run is replaced. A failed run leaves its tag behind — delete it (`git push --delete origin v<version>`) before tagging again, or the next push is rejected as unchanged.

Two things the workflow does that packaging does not

`compile-extensions` and `build-webviews`, in that order, before the gulp packaging task. A module ships as `out/**` (the manifest's `main` is `./out/extension.js`, and `.vscodeignore` excludes `src/**` but not `out/**`), and its webview bundles are esbuild output under that same `out/` — outside VS Code's own media pipeline. A package built without them installs and runs, with an empty surface where each module's page should be.

What a release is not

Nothing is signed: there is no code-signing certificate, so Windows shows "unknown publisher" on first run and macOS needs right-click → Open (or `xattr -d com.apple.quarantine`). Nothing updates itself either — a new version is a new download, not an update. Both are later work, and both are secrets-only changes to the same workflow: a Windows certificate and an Apple Developer ID.

Phases

Phase	Work
1	Rename the repository, add the `upstream` remote, apply the naming
2	`extensions/temco-docs` — the existing extension, ids renamed, gates wired
3	The shell — `src/temco`, the two entry points, `product.json`, menus, welcome
4	Split `temco-ai`, `temco-store`, `temco-ui`, `temco-repositories`, `temco-projects`, `temco-workbench` (**all done** — every module is now one domain, and the workbench is the application)
5	`temco-learn` serves T3000 end to end, then T3001 and T3002